SIMATS ENGINEERING

ASSESSMENT TOOL 2

PSEUDOCODE WRITING TASK — ALGORITHM DESIGN

COURSE NAME: R PROGRAMMING

COURSE CODE: ITA0403

COURSE OUTCOME: CO1

ASSESSMENT WEIGHTAGE: 20%

TOTAL QUESTIONS: 10 | TOTAL MARKS: 100

STUDENT NAME: Sameer Ahmed G

REGISTER NUMBER: 192421154

Code of Conduct

I, Sameer Ahmed G (Register Number: 192421154), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sameer Ahmed G

QUESTION 1: Campus Canteen Billing

1. Step-by-Step Algorithm

1. Start. Store item prices and quantities in numeric vectors of equal length.
2. Multiply corresponding prices and quantities to calculate item bills.
3. Sum item bills to obtain the total.
4. Apply a 10% discount only if the total exceeds ₹500.
5. Subtract the discount and display item bills, total, discount, and payable amount.
6. Stop.

2. Pseudocode — Algorithm Design

```
BEGIN
 INPUT price vector and qty vector
 item_bill ← ELEMENT_WISE_MULTIPLY(price, qty)
 total_bill ← SUM(item_bill)
 IF total_bill > 500 THEN
   discount ← total_bill × 0.10
 ELSE
   discount ← 0
 END IF
 final_amount ← total_bill - discount
 DISPLAY item_bill, total_bill, discount, final_amount
END
```

3. R Code

```
price <- c(120, 80, 150, 60)
qty <- c(2, 1, 2, 1)
item_bill <- price * qty
total_bill <- sum(item_bill)
if (total_bill > 500) {
  discount <- total_bill * 0.10
} else {
  discount <- 0
}
final_amount <- total_bill - discount
cat("Item-wise bills:", item_bill, "\n")
cat("Total bill: Rs.", total_bill, "\n")
cat("Discount: Rs.", discount, "\n")
cat("Final payable amount: Rs.", final_amount, "\n")
```

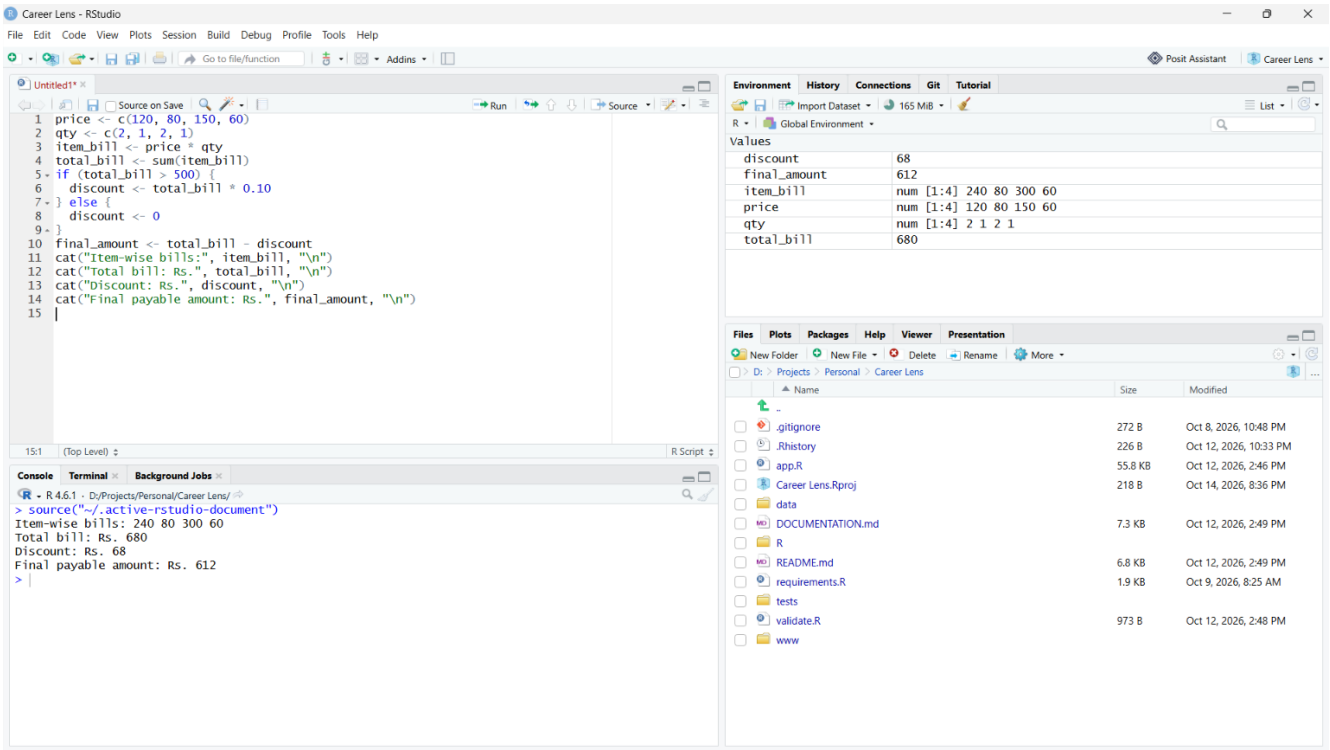
4. Expected Output

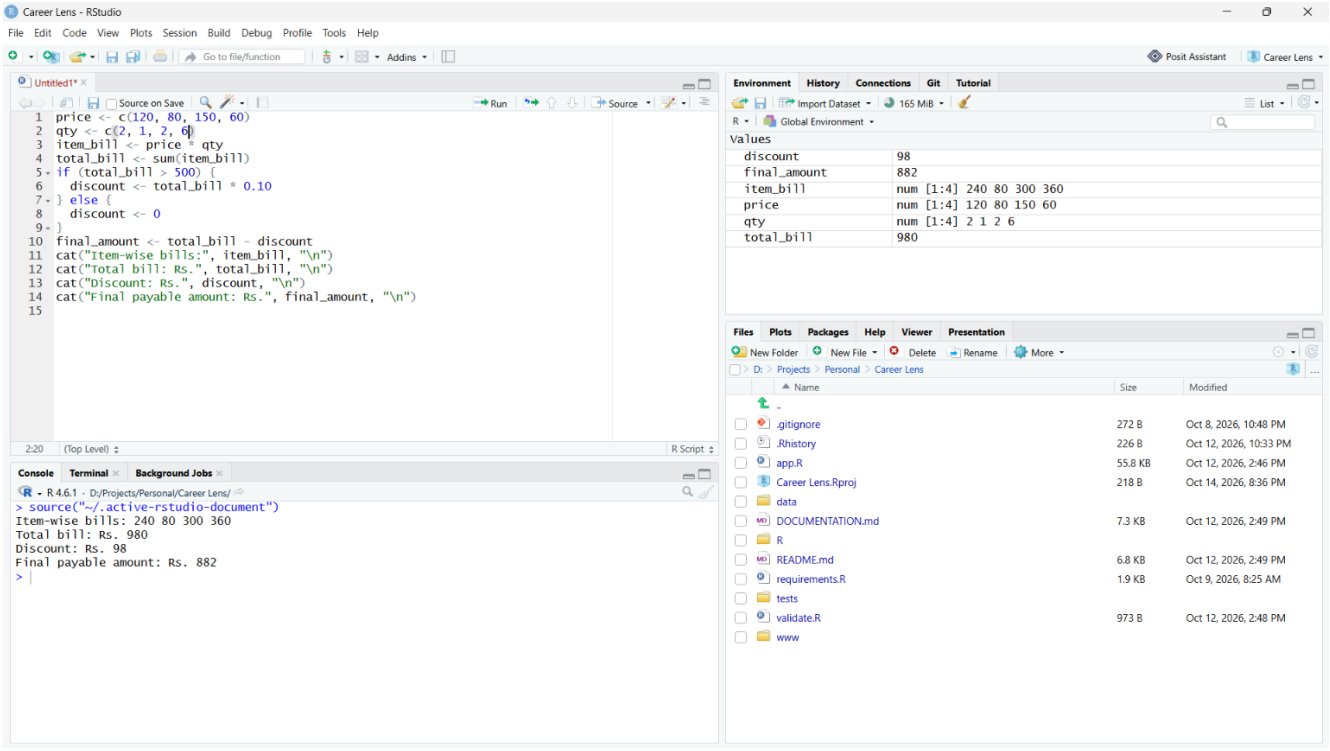
```
Item-wise bills: 240 80 300 60
Total bill: Rs. 680
Discount: Rs. 68
Final payable amount: Rs. 612
```

Result

Item-wise costs and total bill are calculated, and the 10% discount is applied only when the total exceeds ₹500.

RStudio Output Screenshot





QUESTION 2: Student Marks Analyser

1. Step-by-Step Algorithm

7. Start and store the marks of 40 students in a numeric vector.
8. Calculate average, highest mark, and lowest mark.
9. Count marks below 40 as failures and marks of 40 or above as passes.
10. Calculate pass percentage.
11. If more than 80% passed, print “Class Performance: Good”; otherwise print “Needs Improvement”.
12. Display statistics and stop.

2. Pseudocode — Algorithm Design

```
BEGIN
 INPUT marks vector containing 40 marks
 average ← MEAN(marks)
 highest ← MAXIMUM(marks)
 lowest ← MINIMUM(marks)
 failed_count ← COUNT(marks < 40)
 passed_count ← COUNT(marks ≥ 40)
 pass_percentage ← passed_count / LENGTH(marks) × 100
 IF pass_percentage > 80 THEN
   performance ← "Class Performance: Good"
 ELSE
   performance ← "Needs Improvement"
 END IF
 DISPLAY average, highest, lowest, failed_count
 DISPLAY pass_percentage, performance
END
```

3. R Code

```
marks <- c(78,85,92,67,45,38,56,73,89,61,
  95,42,34,76,81,69,55,88,91,47,
  63,72,84,39,58,96,77,68,49,87,
  53,74,82,90,31,65,79,86,44,70)
average <- mean(marks)
highest <- max(marks)
lowest <- min(marks)
failed_count <- sum(marks < 40)
passed_count <- sum(marks >= 40)
pass_percentage <- passed_count / length(marks) * 100
if (pass_percentage > 80) {
  performance <- "Class Performance: Good"
} else {
  performance <- "Needs Improvement"
}
cat("Class average:", round(average, 2), "\n")
cat("Highest mark:", highest, "\n")
cat("Lowest mark:", lowest, "\n")
cat("Failed students:", failed_count, "\n")
cat("Pass percentage:", pass_percentage, "%\n")
cat(performance, "\n")
```

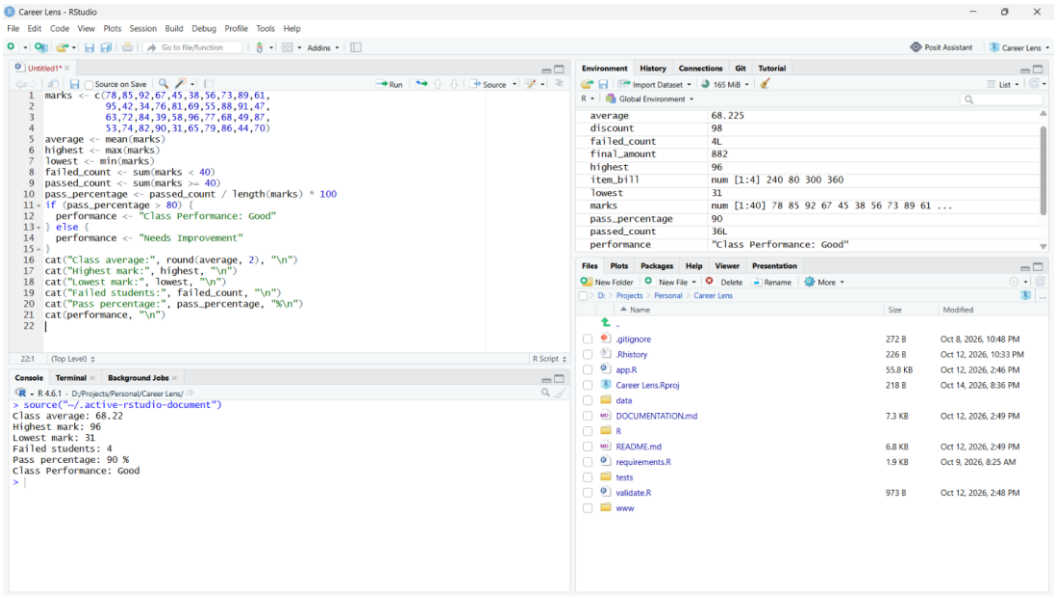
4. Expected Output

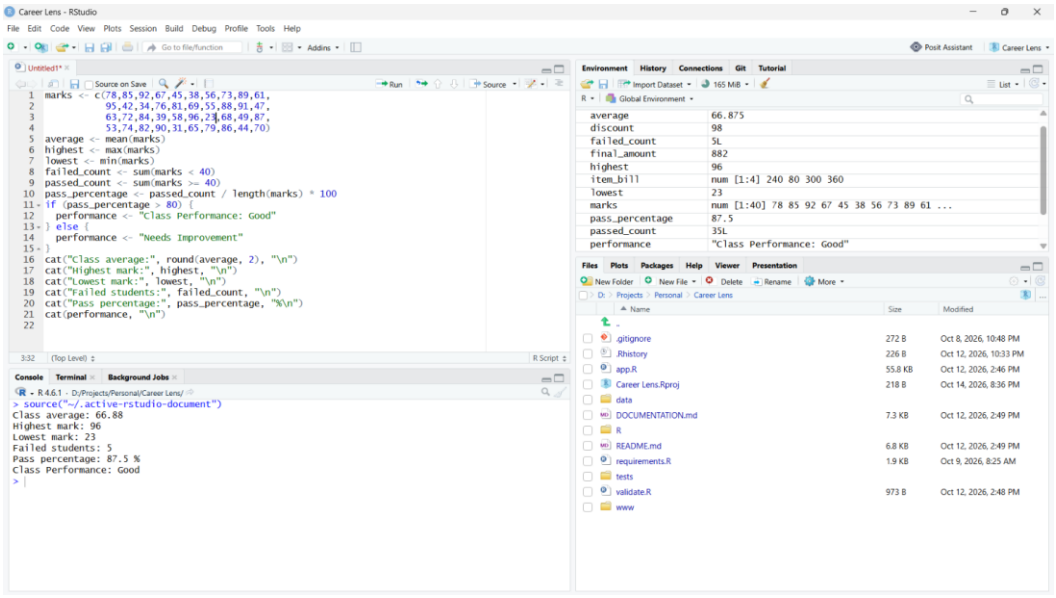
```
Class average: 68.3
Highest mark: 96
Lowest mark: 31
Failed students: 4
Pass percentage: 90 %
Class Performance: Good
```

Result

The program calculates class statistics, counts failures, and classifies performance using the pass percentage.

RStudio Output Screenshot





QUESTION 3: Hostel Room Allocation Matrix

1. Step-by-Step Algorithm

13. Start and create a 4 × 5 matrix where 1 means occupied and 0 means vacant.

14. Count zeros in each row to obtain vacancies per floor.

15. Sum floor-wise vacancies.

16. Find the floor with the largest vacancy count.

17. Display all vacancy results and stop.

2. Pseudocode — Algorithm Design

```
BEGIN
 INPUT occupancy matrix with 4 rows and 5 columns
 FOR floor from 1 to 4
   vacant[floor] ← COUNT(matrix[floor, ] = 0)
 END FOR
 total_vacant ← SUM(vacant)
 maximum_vacancies ← MAXIMUM(vacant)
 floor_number ← POSITION_OF_FIRST_MAXIMUM(vacant)
 DISPLAY vacant, total_vacant, floor_number
END
```

3. R Code

```
rooms <- matrix(c(
  1,1,0,0,1,
  0,1,0,1,0,
  1,1,1,0,1,
  0,0,0,0,1
), nrow=4, byrow=TRUE)
vacant <- rowSums(rooms == 0)
total_vacant <- sum(vacant)
max_vacancies <- max(vacant)
floor_number <- which.max(vacant)
cat("Vacant rooms per floor:", vacant, "\n")
cat("Total vacant rooms:", total_vacant, "\n")
cat("Floor with maximum vacancies:", floor_number, "\n")
cat("Maximum vacancies:", max_vacancies, "\n")
```

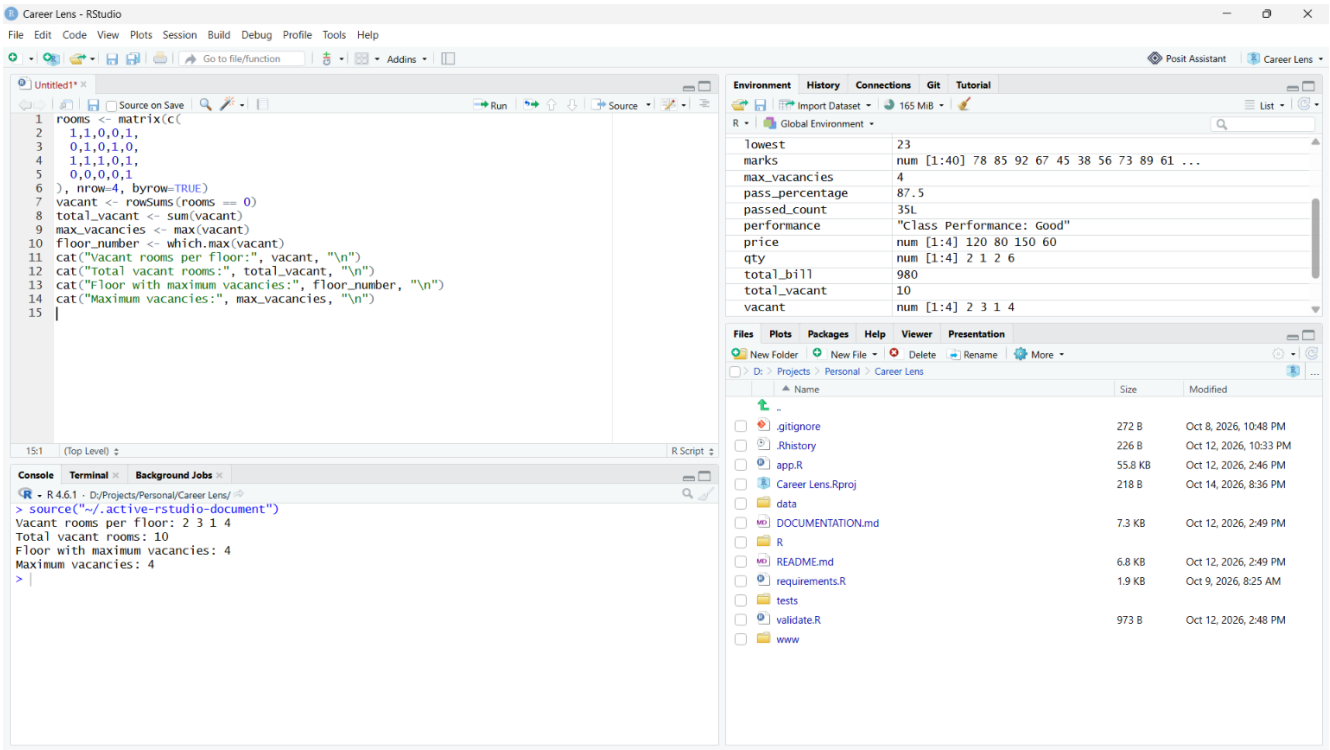
4. Expected Output

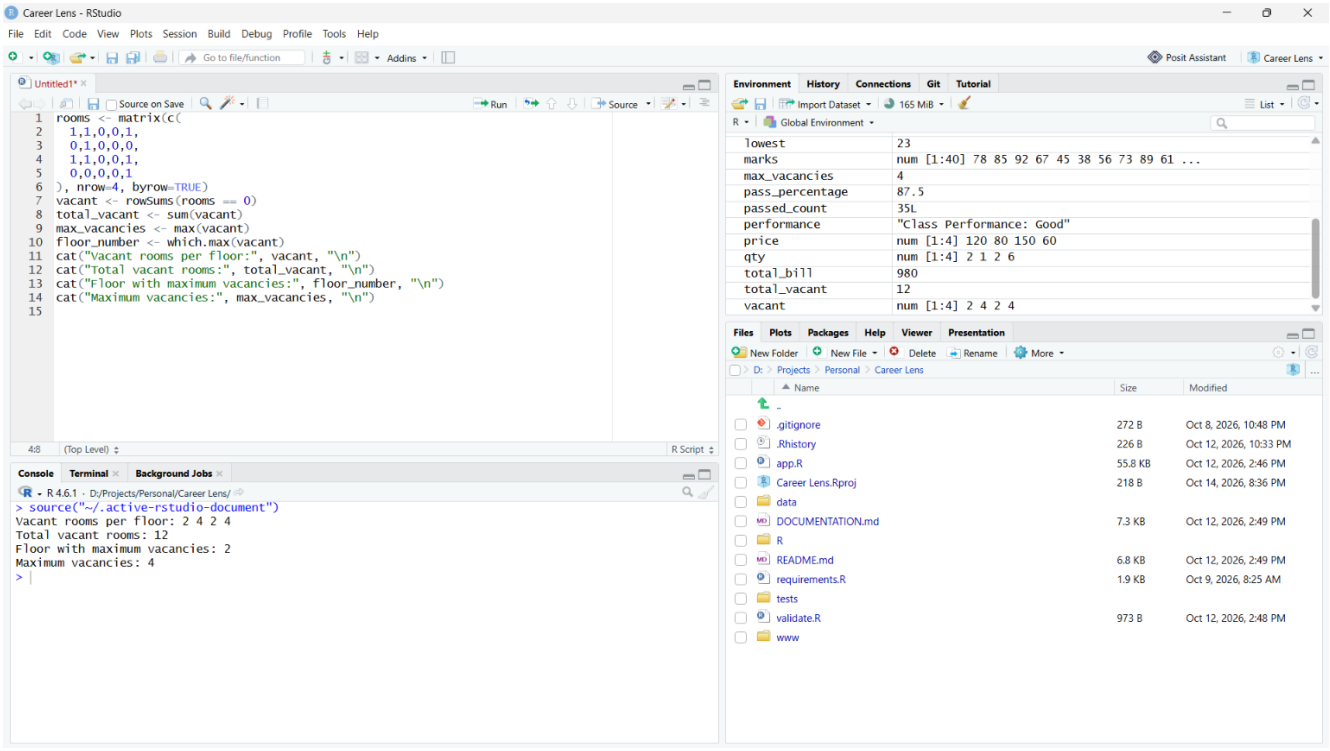
```
Vacant rooms per floor: 2 3 1 4
Total vacant rooms: 10
Floor with maximum vacancies: 4
Maximum vacancies: 4
```

Result

The matrix operations calculate vacancies by floor and identify the floor with the greatest number of vacant rooms.

RStudio Output Screenshot





QUESTION 4: Monthly Electricity Bill Calculator

1. Step-by-Step Algorithm

18. Start and input units consumed.
19. Reject negative consumption.
20. For up to 100 units, charge ₹3 per unit.
21. For 101–200 units, charge ₹300 for the first 100 and ₹5 for each additional unit.
22. Above 200 units, charge ₹300 for the first 100, ₹500 for the next 100, and ₹8 for each additional unit.
23. Add ₹50 fixed charge, display bill summary, and stop.

2. Pseudocode — Algorithm Design

```
BEGIN
 INPUT units
 IF units < 0 THEN
   DISPLAY "Invalid units"
 ELSE
   IF units ≤ 100 THEN
     energy_charge ← units × 3
   ELSE IF units ≤ 200 THEN
     energy_charge ← 100 × 3 + (units - 100) × 5
   ELSE
     energy_charge ← 100 × 3 + 100 × 5 + (units - 200) × 8
   END IF
   total_bill ← energy_charge + 50
   DISPLAY units, energy_charge, fixed charge 50, total_bill
 END IF
END
```

3. R Code

```
units <- 250
if (units < 0) {
  cat("Invalid units\n")
} else {
  if (units <= 100) {
    energy_charge <- units * 3
  } else if (units <= 200) {
    energy_charge <- 100 * 3 + (units - 100) * 5
  } else {
    energy_charge <- 100 * 3 + 100 * 5 + (units - 200) * 8
  }
  fixed_charge <- 50
  total_bill <- energy_charge + fixed_charge
  cat("Units consumed:", units, "\n")
  cat("Energy charge: Rs.", energy_charge, "\n")
  cat("Fixed charge: Rs.", fixed_charge, "\n")
  cat("Total bill: Rs.", total_bill, "\n")
}
```

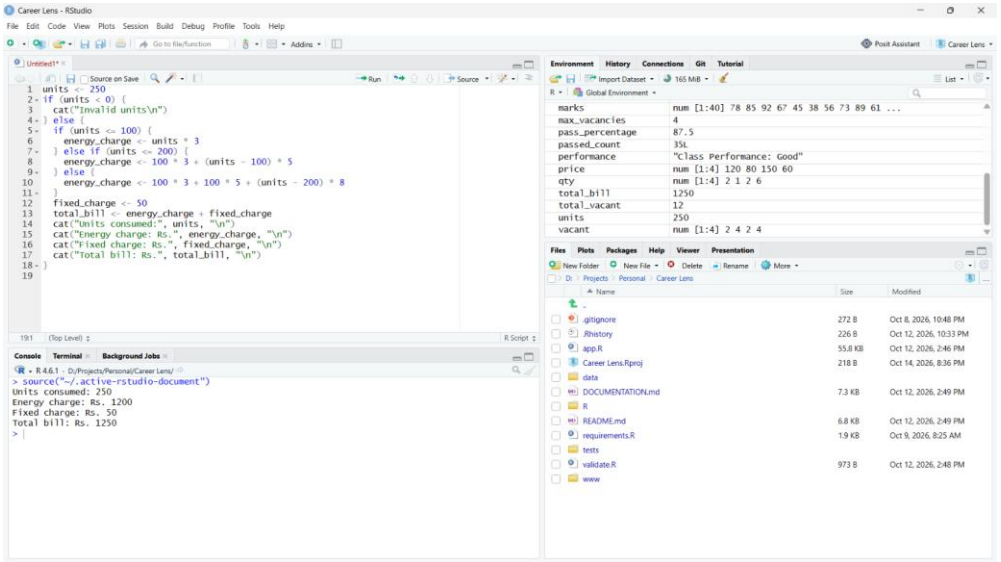
4. Expected Output

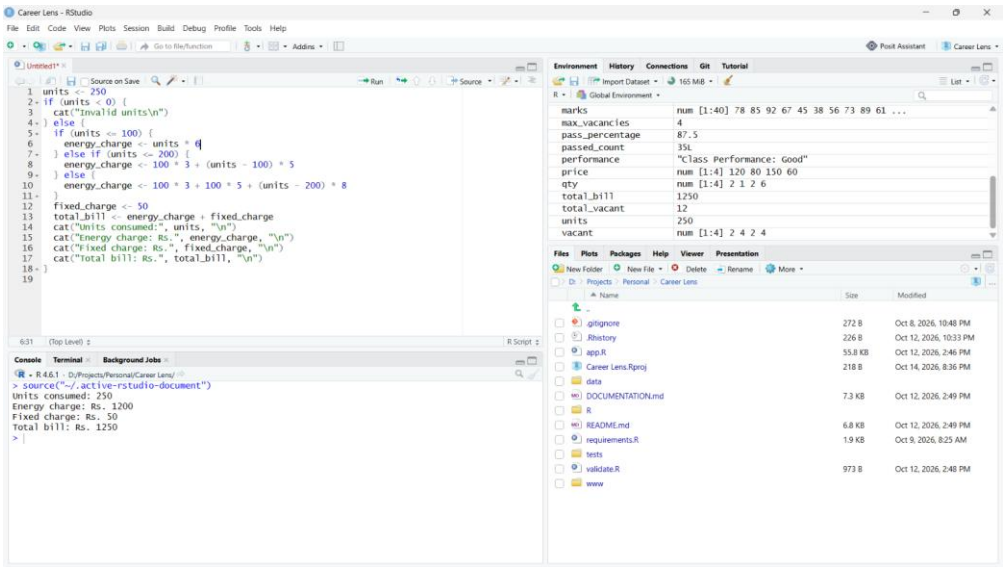
```
Units consumed: 250
Energy charge: Rs. 1200
Fixed charge: Rs. 50
Total bill: Rs. 1250
```

Result

The program applies progressive tariff slabs, validates input, adds the fixed charge, and displays the bill summary.

RStudio Output Screenshot





QUESTION 5: Sales Report Using Matrix Operations

1. Step-by-Step Algorithm

24. Start and create a 3 × 4 sales matrix with products as rows and weeks as columns.
25. Calculate row sums for total sales per product and column sums for weekly totals.
26. Identify the product and week with the maximum totals.
27. Transpose the matrix so weeks become rows.
28. Display totals, best product, best week, and transposed matrix.

2. Pseudocode — Algorithm Design

```
BEGIN
 INPUT sales matrix with 3 product rows and 4 week columns
 product_totals ← ROW_SUMS(sales)
 weekly_totals ← COLUMN_SUMS(sales)
 best_product ← POSITION_OF_FIRST_MAXIMUM(product_totals)
 best_week ← POSITION_OF_FIRST_MAXIMUM(weekly_totals)
 transposed_sales ← TRANSPOSE(sales)
 DISPLAY product_totals, weekly_totals
 DISPLAY best_product, best_week, transposed_sales
END
```

3. R Code

```
sales <- matrix(c(
  120,150,130,170,
  100,180,160,140,
  200,190,210,220
), nrow=3, byrow=TRUE)
rownames(sales) <- c("Product A","Product B","Product C")
colnames(sales) <- c("Week 1","Week 2","Week 3","Week 4")
product_totals <- rowSums(sales)
weekly_totals <- colSums(sales)
best_product <- names(which.max(product_totals))
best_week <- names(which.max(weekly_totals))
transposed_sales <- t(sales)
cat("Product totals:\n"); print(product_totals)
cat("Weekly totals:\n"); print(weekly_totals)
cat("Best product:", best_product, "\n")
cat("Best week:", best_week, "\n")
cat("Transposed matrix:\n"); print(transposed_sales)
```

4. Expected Output

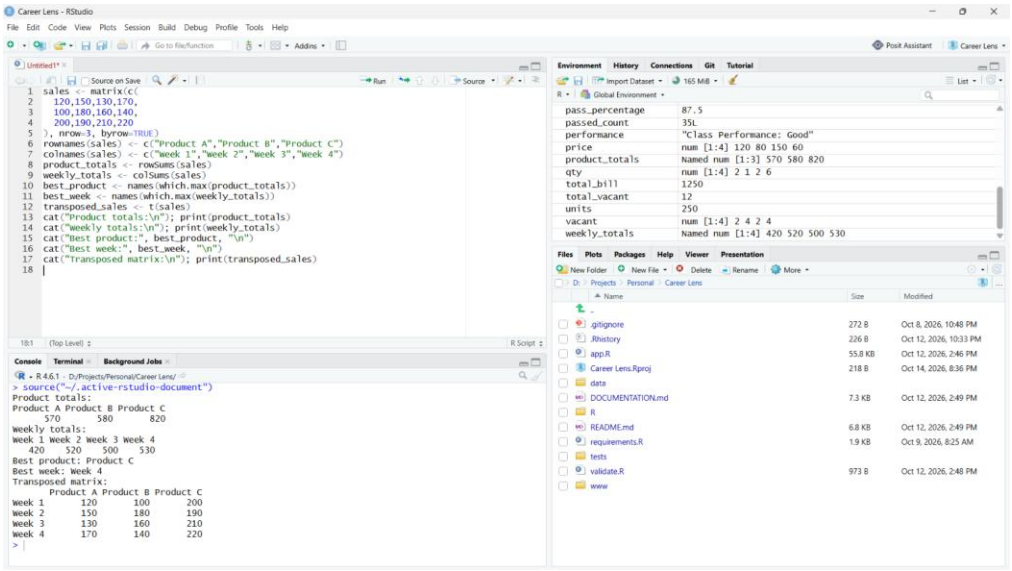
```
Product totals:
Product A Product B Product C
      570       580       820
Weekly totals:
Week 1 Week 2 Week 3 Week 4
   420    520    500    530
Best product: Product C
Best week: Week 4
Transposed matrix:
        Product A Product B Product C
Week 1        120        100        200
Week 2        150        180        190
```

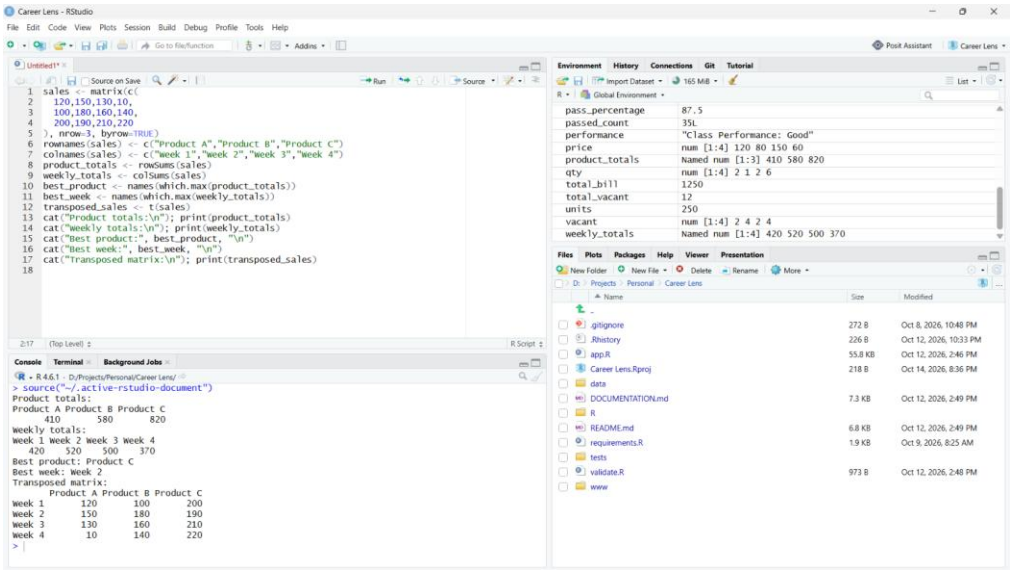
```
Week 3          130          160          210
Week 4          170          140          220
```

Result

Row and column sums identify product and weekly sales performance, while transposition presents weeks as rows.

RStudio Output Screenshot





QUESTION 6: Library Book Management Using Lists

1. Step-by-Step Algorithm

29. Create a list of five book records, each with title, author, year, and availability.

30. Traverse the list and display books published after 2015.

31. Input the title to borrow and search each record.

32. If found and available, update availability to FALSE; otherwise report already unavailable.

33. If no matching title exists, display a not-found message.

34. Stop.

2. Pseudocode — Algorithm Design

```
BEGIN
 CREATE list of 5 book records
 FOR each book
   IF book.year > 2015 THEN DISPLAY book details
   END IF
 END FOR
 INPUT borrow_title
 found ← FALSE
 FOR each book
   IF book.title = borrow_title THEN
     found ← TRUE
     IF book.available = TRUE THEN
       book.available ← FALSE
       DISPLAY "Book borrowed successfully"
     ELSE
       DISPLAY "Book is already unavailable"
     END IF
     EXIT LOOP
   END IF
 END FOR
 IF found = FALSE THEN DISPLAY "Title not found"
 END IF
END
```

3. R Code

```
books <- list(
  list(title="R for Beginners", author="N. Kumar", year=2014, available=TRUE),
  list(title="Data Science Basics", author="A. Rao", year=2018, available=TRUE),
  list(title="Learning R", author="S. Mehta", year=2020, available=TRUE),
  list(title="Modern Computing", author="P. Das", year=2016, available=FALSE),
  list(title="Programming Logic", author="M. Iyer", year=2012, available=TRUE)
)
cat("Books published after 2015:\n")
for (i in seq_along(books)) {
  if (books[[i]]$year > 2015)
    cat(books[[i]]$title, "-", books[[i]]$author, "-", books[[i]]$year, "\n")
}
borrow_title <- "Learning R"
found <- FALSE
for (i in seq_along(books)) {
  if (books[[i]]$title == borrow_title) {
    found <- TRUE
    if (books[[i]]$available) {
```

```
      books[[i]]$available <- FALSE
      cat("Book borrowed successfully:", borrow_title, "\n")
    } else cat("Book is already unavailable:", borrow_title, "\n")
    break
  }
}
if (!found) cat("Title not found:", borrow_title, "\n")
cat("Updated availability:", books[[3]]$available, "\n")
```

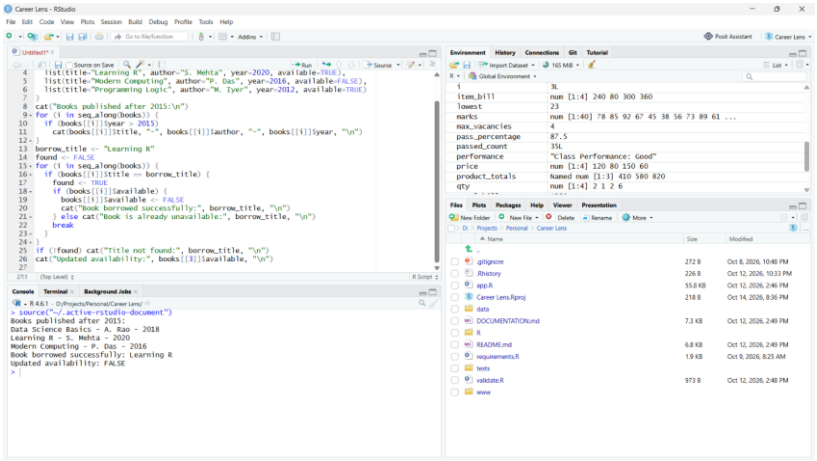
4. Expected Output

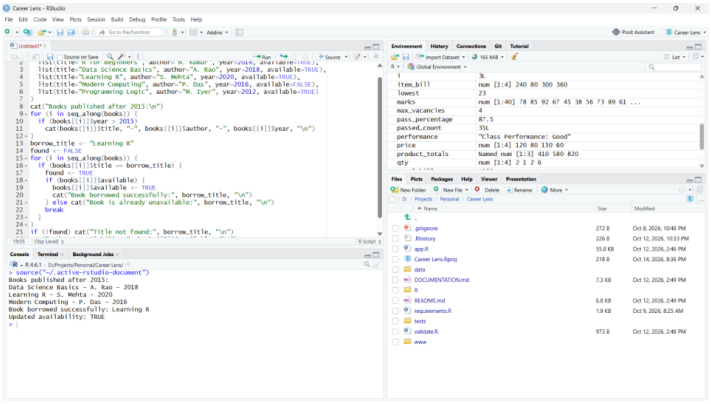
```
Books published after 2015:
Data Science Basics - A. Rao - 2018
Learning R - S. Mehta - 2020
Modern Computing - P. Das - 2016
Book borrowed successfully: Learning R
Updated availability: FALSE
```

Result

The list of lists stores book details, filters books published after 2015, and updates availability with appropriate messages.

RStudio Output Screenshot





QUESTION 7: Temperature Data Cleaning

1. Step-by-Step Algorithm

35. Store 30 readings with missing values represented as NA.
36. Count missing readings and calculate the mean of valid readings only.
37. Replace each NA with that mean.
38. Create a character vector for daily classifications.
39. Classify temperatures below 20°C as Cold, 20–30°C inclusive as Moderate, and above 30°C as Hot.
40. Display missing count, replacement mean, cleaned readings, and classifications.

2. Pseudocode — Algorithm Design

```
BEGIN
 INPUT temperature vector
 missing_count ← COUNT_MISSING(temperature)
 valid_mean ← MEAN(temperature, IGNORE_MISSING = TRUE)
 FOR each position i
   IF temperature[i] IS MISSING THEN temperature[i] ← valid_mean
   END IF
 END FOR
 CREATE empty classification vector
 FOR each temperature x
   IF x < 20 THEN label ← "Cold"
   ELSE IF x ≤ 30 THEN label ← "Moderate"
   ELSE label ← "Hot"
   END IF
   APPEND label to classification
 END FOR
 DISPLAY missing_count, valid_mean, temperature, classification
END
```

3. R Code

```
temperature <- c(18,21,NA,24,31,19,28,NA,33,22,
  17,26,30,NA,35,20,23,16,29,32,
  NA,25,27,34,18,21,30,19,24,31)
missing_count <- sum(is.na(temperature))
valid_mean <- mean(temperature, na.rm=TRUE)
temperature[is.na(temperature)] <- valid_mean
classification <- character(length(temperature))
for (i in seq_along(temperature)) {
  if (temperature[i] < 20) {
    classification[i] <- "Cold"
  } else if (temperature[i] <= 30) {
    classification[i] <- "Moderate"
  } else {
    classification[i] <- "Hot"
  }
}
cat("Missing values:", missing_count, "\n")
cat("Mean used for replacement:", round(valid_mean, 2), "\n")
cat("Cleaned temperatures:\n"); print(round(temperature, 2))
cat("Daily classifications:\n"); print(classification)
```

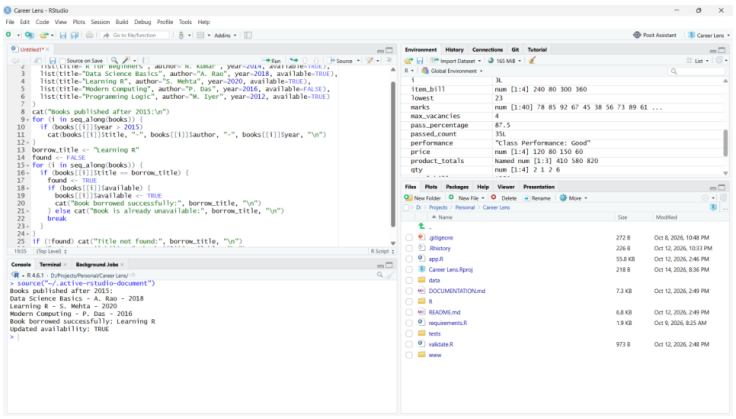
4. Expected Output

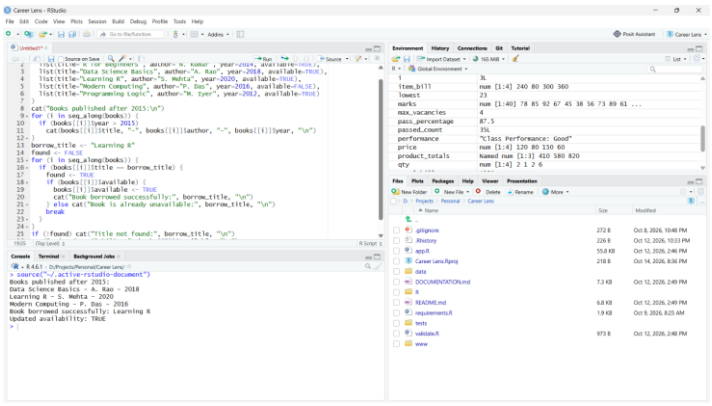
```
Missing values: 4
Mean used for replacement: 25.07
Cleaned temperatures:
 [1] 18.00 21.00 25.07 24.00 31.00 19.00 28.00 25.07 33.00 22.00
[11] 17.00 26.00 30.00 25.07 35.00 20.00 23.00 16.00 29.00 32.00
[21] 25.07 25.00 27.00 34.00 18.00 21.00 30.00 19.00 24.00 31.00
Daily classifications: vector of 30 labels printed by R
```

Result

The program counts missing values, imputes them with the valid-reading mean, and classifies all days into Cold, Moderate, or Hot.

RStudio Output Screenshot





QUESTION 8: Exam Seating Using 3D Arrays

1. Step-by-Step Algorithm

41. Create a 3D array with dimensions 3 blocks × 4 rows × 5 columns; zero denotes an empty seat.
42. Store register numbers at occupied positions.
43. Count non-zero values in each block.
44. Find the block with the highest occupancy.
45. Display the register number at Block 2, Row 3, Column 4.
46. Stop.

2. Pseudocode — Algorithm Design

```
BEGIN
 INPUT seating array with dimensions 3 × 4 × 5
 FOR block b from 1 to 3
   occupied[b] ← COUNT(seating[b, all rows, all columns] ≠ 0)
 END FOR
 highest_block ← POSITION_OF_FIRST_MAXIMUM(occupied)
 seat_value ← seating[2, 3, 4]
 DISPLAY occupied, highest_block, seat_value
END
```

3. R Code

```
seating <- array(0, dim=c(3,4,5))
seating[1,,] <- matrix(c(
  101,102,0,104,105, 106,0,108,109,110,
  111,112,113,0,115, 116,117,118,119,0
), nrow=4, byrow=TRUE)
seating[2,,] <- matrix(c(
  201,202,203,204,0, 206,207,208,209,210,
  211,212,213,214,215, 216,0,218,219,220
), nrow=4, byrow=TRUE)
seating[3,,] <- matrix(c(
  301,0,303,0,305, 0,307,0,309,0,
  311,0,313,0,315, 0,317,0,319,0
), nrow=4, byrow=TRUE)
occupied <- integer(3)
for (b in 1:3) occupied[b] <- sum(seating[b,,] != 0)
highest_block <- which.max(occupied)
seat_value <- seating[2,3,4]
cat("Occupied seats per block:", occupied, "\n")
cat("Block with highest occupancy:", highest_block, "\n")
cat("Block 2, Row 3, Column 4:", seat_value, "\n")
```

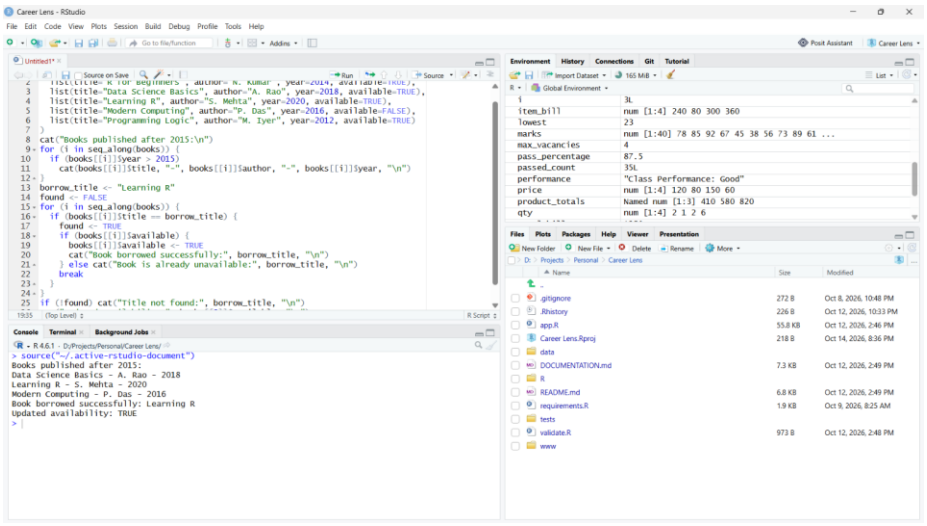
4. Expected Output

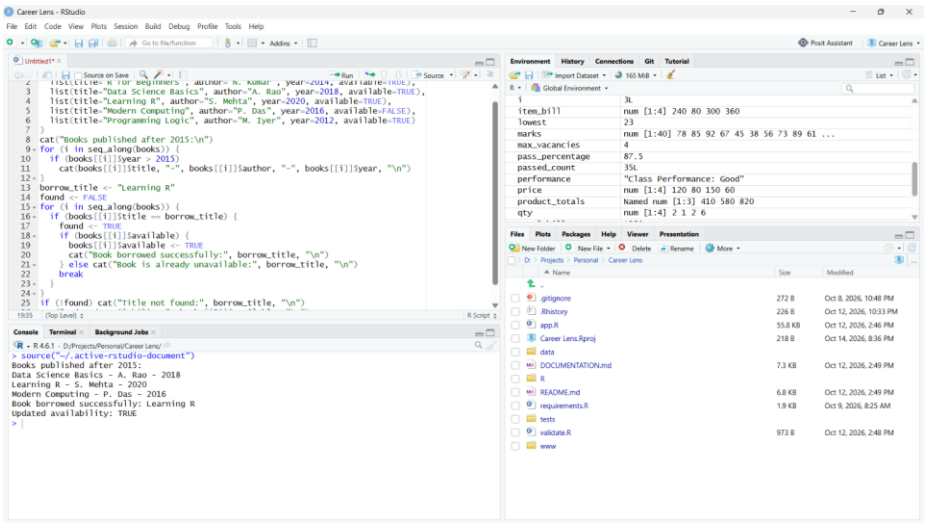
```
Occupied seats per block: 16 18 10
Block with highest occupancy: 2
Block 2, Row 3, Column 4: 214
```

Result

The 3D array is used to count occupied seats by block, find the most occupied block, and retrieve a specific seat register number.

RStudio Output Screenshot





QUESTION 9: Employee Data Frame Processing

1. Step-by-Step Algorithm

47. Create a data frame with EmpID, Name, Department, and Salary.

48. Filter employees in IT whose salary exceeds ₹50,000.

49. Calculate average salary for each department.

50. Create NewSalary from Salary.

51. Increase NewSalary by 10% only for employees earning below ₹30,000.

52. Display filtered employees, departmental averages, and updated data.

2. Pseudocode — Algorithm Design

```
BEGIN
 INPUT employee data frame
 selected ← rows where Department = "IT" AND Salary > 50000
 DISPLAY selected
 departments ← UNIQUE(Department)
 FOR each department d
   average_salary[d] ← MEAN(Salary for employees in d)
 END FOR
 NewSalary ← Salary
 FOR each employee i
   IF Salary[i] < 30000 THEN NewSalary[i] ← Salary[i] × 1.10
   END IF
 END FOR
 ADD NewSalary column
 DISPLAY average_salary and updated data frame
END
```

3. R Code

```
employees <- data.frame(
  EmpID=101:106,
  Name=c("Asha","Bala","Charan","Divya","Eshan","Farah"),
  Department=c("IT","HR","IT","Sales","HR","IT"),
  Salary=c(65000,28000,48000,52000,25000,72000)
)
selected <- subset(employees, Department=="IT" & Salary>50000)
cat("IT employees earning above Rs. 50000:\n"); print(selected)
average_salary <- aggregate(Salary ~ Department, employees, mean)
cat("Average salary by department:\n"); print(average_salary)
employees$NewSalary <- ifelse(employees$Salary < 30000,
                              employees$Salary * 1.10,
                              employees$Salary)
cat("Updated employee data:\n"); print(employees)
```

4. Expected Output

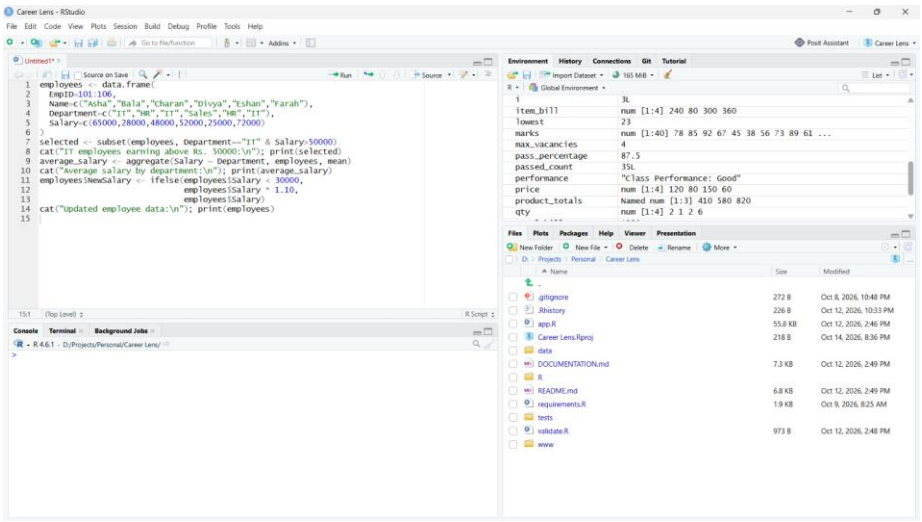
```
IT employees earning above Rs. 50000:
  EmpID  Name Department Salary
1   101  Asha         IT  65000
6   106 Farah         IT  72000
Average salary by department:
  Department   Salary
1          HR 26500.00
```

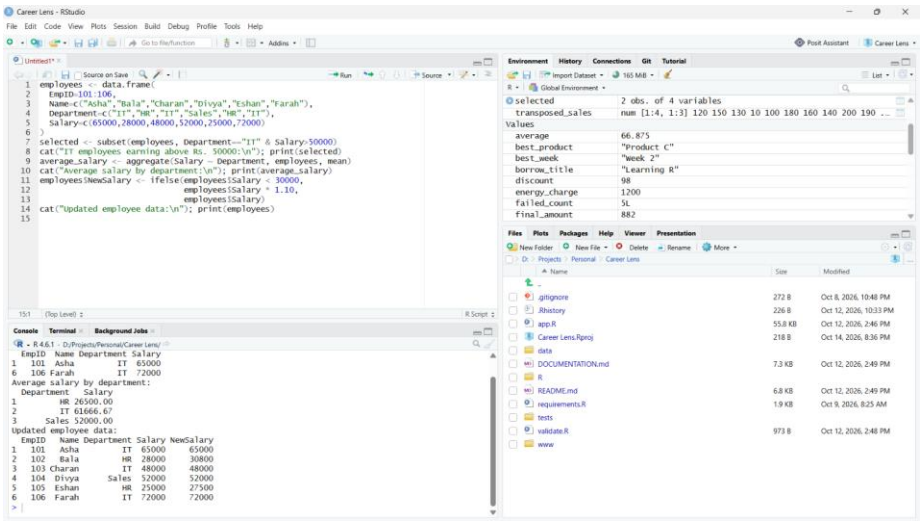
```
2          IT 61666.67
3       Sales 52000.00
NewSalary:
Asha 65000; Bala 30800; Charan 48000; Divya 52000; Eshan 27500; Farah 72000
```

Result

The data frame is filtered, average salaries are grouped by department, and NewSalary applies a 10% raise only below ₹30,000.

RStudio Output Screenshot





QUESTION 10: Prime Number and Fibonacci Series Generator

1. Step-by-Step Algorithm

53. Input N and verify that it is a single positive integer. If invalid, display an error and stop.
54. Create an empty prime vector. Test each candidate from 2 through N for divisors up to its square root using a flag.
55. Append each prime candidate to the vector.
56. Display prime numbers, count, and sum.
57. Generate the first N Fibonacci terms in a vector, using 0 and 1 as the first two terms.
58. Display the Fibonacci vector and stop.

2. Pseudocode — Algorithm Design

```
BEGIN
 INPUT N
 IF N is not a positive integer THEN
   DISPLAY "Invalid input"
   STOP
 END IF
 primes ← EMPTY VECTOR
 FOR candidate from 2 to N
   is_prime ← TRUE
   FOR divisor from 2 to FLOOR(SQUARE_ROOT(candidate))
     IF candidate MOD divisor = 0 THEN
       is_prime ← FALSE
       EXIT INNER LOOP
     END IF
   END FOR
   IF is_prime THEN APPEND candidate to primes
 END FOR
 DISPLAY primes, COUNT(primes), SUM(primes)
 CREATE Fibonacci vector of length N
 FOR position from 1 to N
   IF position = 1 THEN value ← 0
   ELSE IF position = 2 THEN value ← 1
   ELSE value ← previous two terms added
   END IF
   STORE value
 END FOR
 DISPLAY Fibonacci vector
END
```

3. R Code

```
N <- 10
if (length(N) != 1 || is.na(N) || N <= 0 || N != floor(N)) {
  cat("Invalid input: N must be a positive integer.\n")
} else {
  primes <- integer(0)
  if (N >= 2) {
    for (candidate in 2:N) {
      is_prime <- TRUE
      if (candidate >= 4) {
        for (divisor in 2:floor(sqrt(candidate))) {
          if (candidate %% divisor == 0) {
```

```
            is_prime <- FALSE
            break
          }
        }
      }
      if (is_prime) primes <- c(primes, candidate)
    }
  }
  cat("Prime numbers:", primes, "\n")
  cat("Count of primes:", length(primes), "\n")
  cat("Sum of primes:", sum(primes), "\n")
  fibonacci <- numeric(N)
  for (i in seq_len(N)) {
    if (i == 1) fibonacci[i] <- 0
    else if (i == 2) fibonacci[i] <- 1
    else fibonacci[i] <- fibonacci[i-1] + fibonacci[i-2]
  }
  cat("First", N, "Fibonacci terms:", fibonacci, "\n")
}
```

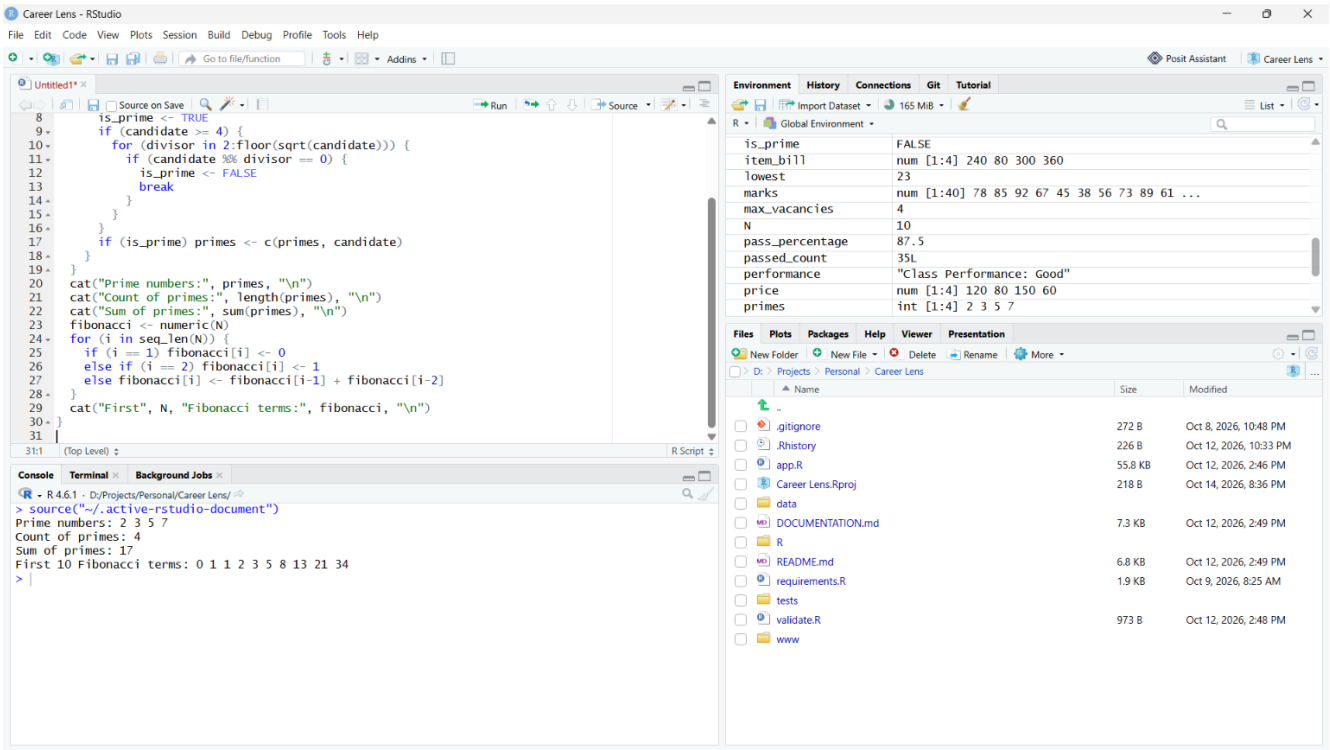
4. Expected Output

```
Prime numbers: 2 3 5 7
Count of primes: 4
Sum of primes: 17
First 10 Fibonacci terms: 0 1 1 2 3 5 8 13 21 34
```

Result

The program validates N, generates prime numbers using nested loops and a flag, computes their count and sum, and stores the first N Fibonacci terms in a vector.

RStudio Output Screenshot



Career Lens - RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Untitled1*
Source on Save
Run
Source
N <- 10
if (length(N) != 1 || is.na(N) || N <= 0 || N != floor(N)) {
cat("Invalid input: N must be a positive integer.\n")
} else {
primes <- integer(0)
if (N >= 2) {
for (candidate in 2:N) {
is_prime <- TRUE
if (candidate >= 4) {
for (divisor in 2:floor(sqrt(candidate))) {
if (candidate %% divisor == 0) {
is_prime <- FALSE
break
}
}
}
if (is_prime) primes <- c(primes, candidate)
}
}
cat("Prime numbers:", primes, "\n")
cat("Count of primes:", length(primes), "\n")
cat("Sum of primes:", sum(primes), "\n")
fibonacci <- numeric(N)
for (i in seq_len(N)) {
31:1 (Top Level) R Script
Console Terminal Background Jobs
R 4.6.1 · D:/Projects/Personal/Career Lens/
> source("~/.active-rstudio-document")
Prime numbers: 2 3 5 7
Count of primes: 4
Sum of primes: 17
First 10 Fibonacci terms: 0 1 1 2 3 5 8 13 21 34